

espol-cuboids
Polarizability Sweep Suite — User Guide

July 10, 2026

Contents

1	Introduction	2
2	Installation	2
3	Project Structure	2
4	Prerequisites	3
5	Running the MoM 3D Sweep	3
5.1	Command-Line Options	3
5.2	Shape Grid	4
5.3	Convergence	4
5.4	Examples	4
5.5	Output	4
6	Running the MoM 2D Sweep	4
6.1	Command-Line Options	4
6.2	Shape Grid	5
6.3	Convergence	5
6.4	Examples	5
6.5	Output	5
7	Running the DDA Sweep	5
7.1	Command-Line Options	5
7.2	Modes	6
7.3	Examples	6
7.4	Output	6
8	Interpreting Results	6
8.1	Depolarization Factors	6
8.2	Convergence Extrapolation	6
8.3	PEC Limit	6
9	Caching	7
10	Running Tests	7

1 Introduction

This suite computes the electrostatic polarizability tensor of dielectric cuboids using three independent numerical methods:

- **MoM 3D** — 3D surface integral equation (Method of Moments) for rectangular cuboids of arbitrary aspect ratio.
- **MoM 2D** — 2D boundary integral equation for infinitely long prisms with rectangular cross-section.
- **DDA** — Discrete Dipole Approximation (volume integral equation) on a cubic lattice inside the cuboid.

All three methods output the normalised polarizability tensor and effective depolarization factors N_x, N_y, N_z for each shape and permittivity. The MoM and DDA solvers also perform convergence extrapolation to the continuum limit.

2 Installation

Listing 1: Clone and install

```
git clone https://github.com/atipaqo/espol-cuboids.git
cd espol-cuboids
pip install -e .
```

Requirements: Python ≥ 3.9 , NumPy ≥ 1.20 , SciPy ≥ 1.7 .

All scripts and library code are accessible after installation. The `espol_cuboids` package can be imported anywhere:

Listing 2: Library usage example

```
from espol_cuboids import (
    gen_mesh_3d_rect, comp_polarizability_3d, Mesh
)
r, n, ds, ne, V = gen_mesh_3d_rect(1.0, 1.0, 1.0, nd=30, octant=True)
P = comp_polarizability_3d(Mesh(r, n, ds, V=V, er=10.0, octant=True))
print(P.xx) # ~2.49
```

3 Project Structure

Listing 3: Key files and directories

```
espol-cuboids/
  pyproject.toml

  src/espol_cuboids/
    __init__.py          # Importable package
    vector_analysis.py   # Public API
    dda_core.py          # V3, D3, Ori, Mesh data structures
    mom2d.py             # DDA solver (FFT-GMRES, convergence)
    mom3d.py             # MoM 2D solver
    gen_dda.py           # MoM 3D solver (LU, GMRES, ACA)
    gen_mom_2d.py        # DDA batch runner (cache, convergence fit)
    gen_mom_3d.py        # MoM 2D batch runner
    gen_mesh_2d_rect.py  # MoM 3D batch runner
                      # 2D perimeter mesh generator
```

```

gen_mesh_3d_rect.py          # 3D surface mesh generator

examples/
  basic_usage.py             # Single-point demonstration
  run_mom3d_sweep.py         # 3D MoM sweep entry point
  run_mom2d_sweep.py         # 2D MoM sweep entry point
  run_dda_sweep.py           # DDA sweep entry point

tests/
  test_vector.py             # V3, D3, Mesh unit tests
  test_dda_core.py           # DDA correctness & regression
  test_mom2d.py              # 2D MoM mesh, solver, regression
  test_mom3d.py              # 3D MoM mesh, solver, ACA, convergence

data/                        # Caches & CSV output (auto-created)
  csvdata/
  _mom3d_cache/
  _mom2d_cache/
  _dda_cache/

docs/                        # Documentation
  user_guide.pdf

```

4 Prerequisites

- Python 3.9+ with NumPy and SciPy (installed automatically by `pip install -e .`).
- Cached results are stored in `data/_mom3d_cache/`, `data/_mom2d_cache/`, and `data/_dda_cache/`. Delete these directories to force recomputation.

5 Running the MoM 3D Sweep

5.1 Command-Line Options

```
python examples/run_mom3d_sweep.py [--er ER_LIST] [--solver direct|aca] [--dry-run ]
```

Flag	Values	Description
<code>--er</code>	Comma-separated floats	Relative permittivities to sweep. Default: 10,20,100,1e10 (where $1e10 \approx \text{PEC}$).
<code>--solver</code>	<code>direct</code> (default), <code>aca</code>	Solver backend. <code>direct</code> uses LU factorization; <code>aca</code> uses ACA-compressed H-matrix + GMRES (recommended for large meshes, $N > 2000$).
<code>--dry-run</code>	(no value)	Print the shapes, ϵ_r values, and n_d sequences that would be computed, then exit.

5.2 Shape Grid

The sweep covers 15 aspect ratios $(u, v) = (l_y/l_x, l_z/l_x)$:

(1.0, 1.0), (1.0, 0.7), (1.0, 0.5), (1.0, 0.3), (1.0, 0.2), (0.7, 0.7),
 (0.7, 0.5), (0.7, 0.3), (0.7, 0.2), (0.5, 0.5), (0.5, 0.3), (0.5, 0.2),
 (0.3, 0.3), (0.3, 0.2), (0.2, 0.2)

with $l_x = 1$ fixed.

5.3 Convergence

For each shape, the solver runs 4–6 mesh resolutions (n_d) chosen adaptively by `adaptive_nd()`. A power-law fit $P(n_d) = P_\infty + C n_d^{-\beta}$ extrapolates to the continuum limit. Individual solves and converged fits are cached on disk.

5.4 Examples

```
# Preview what will be computed
python examples/run_mom3d_sweep.py --dry-run

# Sweep with default permittivities (direct LU)
python examples/run_mom3d_sweep.py

# Single permittivity with ACA solver
python examples/run_mom3d_sweep.py --er 10 --solver aca

# Custom permittivity range
python examples/run_mom3d_sweep.py --er 1.5,3,6,12,100
```

5.5 Output

Results are printed to stdout as the sweep progresses. The depolarization factors N_x, N_y, N_z and their sum $N_\Sigma = N_x + N_y + N_z$ are shown for each shape- ε_r pair.

For bulk data export, use the standalone `gen_mom_3d.main()` function (run `python -c "from espol_cuboids.gen_mom_3d import main; main()" --dry-run`).

6 Running the MoM 2D Sweep

6.1 Command-Line Options

```
python examples/run_mom2d_sweep.py [--er ER_LIST] [--dry-run]
```

Flag	Values	Description
<code>--er</code>	Comma-separated floats	Relative permittivities. Default: 0.7,1.5,3,6,1e3,1e5.
<code>--dry-run</code>	(no value)	Print the shapes and n_d sequences that would be computed, then exit.

The 2D solver uses only the direct LU backend (no ACA/GMRES option), as problem sizes remain small ($n \lesssim 500$).

6.2 Shape Grid

23 aspect ratios $r = l_y/l_x$ with $l_x = 1$:

0.1, 0.15, 0.2, 0.3, 0.35, 0.4, 0.5, 0.6, 0.667, 0.8, 0.9, 1.0, 1.1, 1.25, 1.5, 1.75, 2.0, 2.5, 3.0, 3.33, 5.0, 7.0, 10.0

6.3 Convergence

n_d sequences are chosen adaptively. A power-law fit extrapolates to $n_d \rightarrow \infty$. Individual solves and fits are cached in `data/_mom2d_cache/`.

6.4 Examples

```
# Preview
python examples/run_mom2d_sweep.py --dry-run

# Full sweep
python examples/run_mom2d_sweep.py

# Custom permittivities
python examples/run_mom2d_sweep.py --er 2,5,10
```

6.5 Output

For each shape- ϵ_r pair, the normalized depolarization factors

$$N_x = \frac{1}{P_{xx}/A}, \quad N_y = \frac{1}{P_{yy}/A}, \quad N_\Sigma = N_x + N_y$$

are printed to stdout. Results are cached on disk for reuse.

7 Running the DDA Sweep

7.1 Command-Line Options

```
python examples/run_dda_sweep.py --mode fixed|converge [--er ER_LIST] [--dry-run]
```

Flag	Values	Description
<code>--mode</code>	<code>fixed</code> , <code>converge</code>	fixed : single lattice spacing $d = s_{\min}/10$. converge : multiple d levels with $d \rightarrow 0$ fit.
<code>--er</code>	Comma-separated floats	Relative permittivities. Default: 0.5,1.25,1.5,1.75,2,3,5,8,10,12.5.
<code>--dry-run</code>	(no value)	Print the shapes and lattice parameters, then exit.

The DDA sweep shares the same 15-shape grid as the MoM 3D sweep (Section 5.2). The FFT-accelerated GMRES backend is used for all solves ($N \gtrsim 1000$).

7.2 Modes

fixed Each shape is solved at a single lattice spacing $d = \min(l_y, l_z)/10$. Fast, suitable for quick surveys. Output: `data/csvdata/dda3d.csv`.

converge Each shape is solved at 4–5 values of d (chosen so that $N \leq 20\,000$ dipoles per solve).

A power-law $P(d) = P_\infty + C d^\beta$ fit yields the continuum limit. Output: `data/csvdata/dda_convergence.csv` (individual solves) and `data/csvdata/dda_convergence_cnv.csv` (extrapolated values).

7.3 Examples

```
# Preview fixed mode
python examples/run_dda_sweep.py --mode fixed --dry-run

# Quick fixed-d sweep (all er, all shapes)
python examples/run_dda_sweep.py --mode fixed

# Converge mode for er=10 only
python examples/run_dda_sweep.py --mode converge --er 10

# Converge mode with multiple er values
python examples/run_dda_sweep.py --mode converge --er 2,5,10
```

7.4 Output

Depolarization factors N_x, N_y, N_z and their sum are printed per shape- ε_r - d combination. CSV files are written to the `data/csvdata/` directory.

8 Interpreting Results

8.1 Depolarization Factors

The depolarization factor N_i (for axis i) relates the internal field to the applied field in an ellipsoid:

$$E_i^{\text{int}} = E_i^{\text{app}} - \frac{N_i}{\varepsilon_0} P_i$$

For cuboids, N_i is shape- and permittivity-dependent (unlike ellipsoids, where it depends only on shape). The sum rule

$$N_x + N_y + N_z = 1 \quad (3\text{D}), \quad N_x + N_y \approx 1 \quad (2\text{D})$$

holds exactly for ellipsoids and approximately for cuboids at moderate ε_r .

8.2 Convergence Extrapolation

The power-law fit $P(d) = P_\infty + C d^\beta$ is justified by the leading discretization-error term for smooth geometries. The exponent β is typically 1–2 for the MoM and 1–3 for the DDA. Fits are monitored; if the residual is large or β falls outside expected bounds, the finest-resolution value is used as a fallback.

8.3 PEC Limit

For the DDA, the PEC limit is obtained by fitting $\alpha(\varepsilon_r) = \alpha_{\text{PEC}} + A/\varepsilon_r$. For the MoM, $\varepsilon_r = 10^{10}$ is used as a proxy (the system matrix becomes nearly singular for true PEC, but $\varepsilon_r = 10^{10}$ gives the correct limit to ~ 4 significant digits).

9 Caching

All three sweeps cache individual solves on disk:

- **MoM 3D:** `data/_mom3d_cache/` — keyed by ε_r , shape, n_d , β , and solver type (direct/ACA).
- **MoM 2D:** `data/_mom2d_cache/` — keyed by ε_r , shape, n_d , β .
- **DDA:** `data/_dda_cache/` — keyed by ε_r , shape, and lattice spacing d .

Cache keys include version tags; changing the implementation version automatically invalidates stale cache entries. To force a full recomputation, delete the relevant cache directory.

10 Running Tests

The test suite is run with `pytest` from the `espol-cuboids/` directory:

```
python -m pytest tests/ -v          # all 54 tests
python -m pytest tests/ -v -m "not slow"  # skip full MoM/DDA solves
```

Tests cover vector operations, mesh generation, MoM 2D/3D solvers, the DDA solver, the ACA backend, convergence extrapolation, and regression against known values.